Adrian Martinez Series de tiempo

Pregunta 18

Link GITHUB

https://github.com/A01178406/Examen-final-18.git

Link Colab

https://colab.research.google.com/drive/1mzngu0LCl_fH-8GFRdxojnWgcVsX8vK1?usp=sharing

Link Grok

https://grok.com/share/c2hhcmQtMg%3D%3D_aaf83a79-f98c-4019-b0e9-061fcd09f4a2

HTML

```python
# -*- coding: utf-8 -*-
"""Pregunta 18

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1mzngu0LCl_fH-8GFRdxojnWgcVsX8vK1
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox

# Create DataFrame with complete MONEY data
data = pd.DataFrame({
    'TIME': range(1, 490),
    'MONEY': [
        138.89, 139.39, 139.74, 139.69, 140.68, 141.17, 141.7, 141.9, 141.01, 140.47,
        140.38, 139.95, 139.98, 139.87, 139.75, 139.56, 139.61, 139.58, 140.18, 141.31,
        141.18, 140.92, 140.86, 140.69, 141.06, 141.6, 141.87, 142.13, 142.66, 142.88,
        142.92, 143.49, 143.78, 144.14, 144.76, 145.2, 145.24, 145.66, 145.96, 146.4,
        146.84, 146.58, 146.46, 146.57, 146.3, 146.71, 147.29, 147.82, 148.26, 148.9,
        149.17, 149.7, 150.39, 150.43, 151.34, 151.78, 151.98, 152.55, 153.65, 153.29,
        153.74, 154.31, 154.48, 154.77, 155.33, 155.62, 156.8, 157.82, 158.75, 159.24,
        159.96, 160.3, 160.71, 160.94, 161.47, 162.03, 161.7, 162.19, 163.05, 163.68,
        164.85, 165.97, 166.71, 167.85, 169.08, 169.62, 170.51, 171.81, 171.33, 171.57,
        170.31, 170.81, 171.97, 171.16, 171.38, 172.03, 171.86, 172.99, 174.81, 174.17,
        175.68, 177.02, 178.13, 179.71, 180.68, 181.64, 182.38, 183.26, 184.33, 184.71,
        185.47, 186.6, 187.99, 189.42, 190.49, 191.84, 192.74, 194.02, 196.02, 197.41,
        198.69, 199.35, 200.02, 200.71, 200.81, 201.27, 201.66, 201.73, 202.1, 202.9,
        203.57, 203.88, 206.22, 205, 205.75, 206.72, 207.22, 207.54, 207.98, 209.93,
        211.8, 212.88, 213.66, 214.41, 215.54, 217.42, 218.77, 220, 222.02, 223.45,
        224.85, 225.58, 226.47, 227.16, 227.76, 228.32, 230.09, 232.32, 234.3, 235.58,
        235.89, 236.62, 238.79, 240.93, 243.18, 245.02, 246.41, 249.25, 251.47, 252.15,
        251.67, 252.74, 254.89, 256.69, 257.54, 257.76, 257.86, 259.04, 260.98, 262.88,
        263.76, 265.31, 266.68, 267.2, 267.56, 268.44, 269.27, 270.12, 271.05, 272.35,
        273.71, 274.2, 273.9, 275, 276.42, 276.17, 279.2, 282.43, 283.68, 284.15, 285.69,
        285.39, 286.83, 287.07, 288.42, 290.76, 292.7, 294.66, 295.93, 296.16, 297.2,
        299.05, 299.67, 302.04, 303.59, 306.25, 308.26, 311.54, 313.94, 316.02, 317.19,
        318.71, 320.19, 322.27, 324.48, 326.4, 328.64, 330.87, 334.4, 335.3, 336.96,
        339.92, 344.86, 346.8, 347.63, 349.66, 352.26, 353.35, 355.41, 357.28, 358.6,
        359.91, 362.45, 368.05, 369.59, 373.34, 377.21, 378.82, 379.28, 380.87, 380.81,
        381.77, 385.85, 389.7, 388.13, 383.44, 384.6, 389.46, 394.91, 400.06, 405.36,
        409.06, 410.37, 408.06, 410.83, 414.38, 418.69, 427.06, 424.43, 425.5, 427.9,
        427.85, 427.46, 428.45, 430.88, 436.17, 442.13, 441.49, 442.37, 446.78, 446.53,
```

```
        447.89, 449.09, 452.49, 457.5, 464.57, 471.12, 474.3, 476.68, 483.85, 490.18,
        492.77, 499.78, 504.35, 508.96, 511.6, 513.41, 517.21, 518.53, 520.79, 524.4,
        526.99, 530.78, 534.03, 536.59, 540.54, 542.13, 542.39, 543.86, 543.87, 547.32,
        551.19, 555.66, 562.48, 565.74, 569.55, 575.07, 583.17, 590.82, 598.06, 604.47,
        607.91, 611.83, 619.36, 620.4, 624.14, 632.81, 640.35, 652.01, 661.52, 672.2,
        680.77, 688.51, 695.26, 705.24, 724.28, 729.34, 729.84, 733.01, 743.39, 746,
        743.72, 744.96, 746.96, 748.66, 756.5, 752.83, 749.68, 755.55, 757.07, 761.18,
        767.57, 771.68, 779.1, 783.4, 785.08, 784.82, 783.63, 784.46, 786.26, 784.92,
        783.4, 782.74, 778.82, 774.79, 774.22, 779.71, 781.14, 782.2, 787.05, 787.95,
        792.57, 794.93, 797.65, 801.25, 806.24, 804.36, 810.33, 811.8, 817.85, 821.83,
        820.3, 822.06, 824.56, 826.73, 832.4, 838.62, 842.73, 848.96, 858.33, 862.95,
        868.65, 871.56, 878.4, 887.95, 896.7, 910.49, 925.13, 936, 943.89, 950.78,
        954.71, 964.6, 975.71, 988.84, 1004.34, 1016.04, 1024.45, 1030.9, 1033.15,
        1037.99, 1047.47, 1066.22, 1075.61, 1085.88, 1095.56, 1105.43, 1113.8,
        1123.9, 1129.31, 1132.2, 1136.13, 1139.91, 1141.42, 1142.85, 1145.65,
        1151.49, 1151.39, 1152.44, 1150.41, 1150.44, 1149.75, 1150.64, 1146.74,
        1146.52, 1149.48, 1144.65, 1144.24, 1146.5, 1146.1, 1142.27, 1136.43,
        1133.55, 1126.73, 1122.58, 1117.53, 1122.59, 1124.52, 1116.3, 1115.47,
        1112.34, 1102.18, 1095.61, 1082.56, 1080.49, 1081.34, 1080.52, 1076.2,
        1072.42, 1067.45, 1063.37, 1065.99, 1067.57, 1072.08, 1064.82, 1062.06,
        1067.53, 1074.87, 1073.81, 1076.02, 1080.65, 1082.09, 1078.17, 1077.78,
        1075.37, 1072.21, 1074.65, 1080.4, 1088.96, 1093.35, 1091, 1092.65,
        1102.01, 1108.4, 1104.75, 1101.11, 1099.53, 1102.4, 1093.46
    ]
})

# Log-transform the MONEY column
log_money = np.log(data['MONEY'])

# Step 1: Identification
# Plot the log-transformed series
plt.figure(figsize=(10, 6))
plt.plot(log_money, label='Log M1 Money Supply')
plt.title('Log-Transformed M1 Money Supply (1951-1999)')
plt.xlabel('Time (Months from Jan 1951)')
plt.ylabel('Log M1')
plt.legend()
plt.show()

# Augmented Dickey-Fuller test for stationarity
def adf_test(series, title=''):
    result = adfuller(series, autolag='AIC')
    print(f'ADF Test for {title}:')
    print(f'ADF Statistic: {result[0]}')
    print(f'p-value: {result[1]}')
    print(f'Critical Values: {result[4]}')
    if result[1] <= 0.05:
        print("Series is stationary (reject null hypothesis)\n")
    else:
```

```
    else:
        print("Series is non-stationary (fail to reject null hypothesis)\n")

adf_test(log_money, 'Log M1 Money Supply')

# Difference the series if non-stationary (d=1)
diff_log_money = log_money.diff().dropna()
adf_test(diff_log_money, 'First-Differenced Log M1')

# Plot ACF and PACF for differenced series
plt.figure(figsize=(12, 6))
plt.subplot(121)
plot_acf(diff_log_money, lags=40, ax=plt.gca(), title='ACF of Differenced Log M1')
plt.subplot(122)
plot_pacf(diff_log_money, lags=40, ax=plt.gca(), title='PACF of Differenced Log M1')
plt.tight_layout()
plt.show()

# Step 2: Estimation
# Try ARIMA(1,1,1) based on initial ACF/PACF inspection (adjust as needed)
model = ARIMA(log_money, order=(1, 1, 1))
results = model.fit()

# Print model summary
print(results.summary())

# Step 3: Diagnostic Checking
# Plot residuals
residuals = results.resid
plt.figure(figsize=(10, 6))
plt.plot(residuals)
plt.title('Residuals of ARIMA(1,1,1) Model')
plt.xlabel('Time')
plt.ylabel('Residuals')
plt.show()

# ACF of residuals
plt.figure(figsize=(10, 6))
plot_acf(residuals, lags=40, title='ACF of Residuals')
plt.show()

# Ljung-Box test for residual autocorrelation
lb_test = acorr_ljungbox(residuals, lags=[10], return_df=True)
print('Ljung-Box Test Results:')
print(lb_test)

# Step 4: Forecasting
# Generate 12-month forecasts
forecast = results.forecast(steps=12)
forecast_index = range(len(log_money), len(log_money) + 12)


# Plot actual data and forecasts
plt.figure(figsize=(10, 6))
plt.plot(log_money, label='Log M1 Money Supply')
plt.plot(forecast_index, forecast, label='12-Month Forecast', color='red')
plt.title('ARIMA(1,1,1) Forecast for Log M1 Money Supply')
plt.xlabel('Time (Months from Jan 1951)')
plt.ylabel('Log M1')
plt.legend()
plt.show()
```

Interpretations

The Python code implements the Box-Jenkins methodology to fit an ARIMA model to the log-transformed M1 money supply data (1951–1999), serving as a substitute for log DPI as specified in Gujarati's Basic Econometrics. The process begins with a logarithmic transformation of the M1 series to stabilize variance, though the plotted

series likely exhibits a persistent upward trend, suggesting non-stationarity, which an Augmented Dickey-Fuller (ADF) test would confirm with a high p-value. First differencing renders the series stationary, typically evidenced by a low ADF p-value, indicating an integration order of d=1. Autocorrelation (ACF) and partial autocorrelation (PACF) plots of the differenced series inform the selection of ARIMA(1,1,1), though these plots may suggest alternative AR (p) or MA (q) orders. The model is estimated, yielding coefficients and fit metrics such as AIC, with its adequacy assessed through residual diagnostics: residuals should resemble white noise, their ACF plot should show no significant lags, and the Ljung-Box test should produce p-values above 0.05 to confirm no residual autocorrelation. A 12-month forecast is generated, likely showing a modest upward trend that stabilizes due to differencing, though its accuracy depends on model fit. Substituting M1 for DPI may introduce discrepancies, as DPI could exhibit distinct dynamics or seasonality. Executing the code and analyzing outputs ADF results, ACF/PACF plots, and diagnostics is essential to refine the model, potentially adjusting parameters or incorporating seasonal ARIMA if monthly patterns are detected, ensuring a robust analysis of the economic time series.